Process Management, System Info & Services

Section 9 — Detailed Command Reference | nezuko homelab

9.1 Key Concepts

What is a Process?

A **process** is a running instance of a program. Every time you run a command, the OS creates a process, executes it, and (usually) destroys it when done. Long-running programs like web servers or SSH daemons stay alive as permanent processes.

Property	What it means
PID	Process ID — unique number identifying this process on this system
PPID	Parent Process ID — the process that spawned this one
Owner	The user account running this process
State	Current state: Running (R), Sleeping (S), Stopped (T), Zombie (Z)
TTY	Terminal attached to the process — ? means no terminal (background daemon)

What is a Daemon?

A **daemon** is a background process that runs continuously — like sshd (SSH server), cron (scheduler), or dockerd (Docker). Daemons typically have no terminal (TTY shows ?), start at boot, and are managed by systemd. The 'd' at the end of a process name usually means daemon — sshd, crond, httpd.

What are Signals?

Signals are messages sent to a process to control its behaviour. Think of them like tapping someone on the shoulder (SIGTERM) vs physically removing them from the room (SIGKILL).

Signal	Number	What it does	Can process ignore it?
SIGTERM	15	Polite request to terminate — process can clean up first	Yes
SIGKILL	9	Force kill — immediate termination, no cleanup	No — always works
SIGINT	2	Interrupt — same as pressing Ctrl+C	Yes
SIGHUP	1	Hangup — often used to reload config without restart	Yes

Command: ps

ps

Report a snapshot of current processes

What it does:
ps takes a snapshot of running processes at the moment you run it — unlike top/htop it does not update in real time. It is the standard tool for scripting and quick checks. On its own, ps only shows processes attached to your current terminal.

Common options:

Option	What it does
ps	Show processes in your current terminal session only
ps aux	Show ALL processes on the system from ALL users — the most used form
ps aux --sort=-%mem	Show all processes sorted by memory usage, highest first
ps aux --sort=-%cpu	Show all processes sorted by CPU usage, highest first
ps -p PID	Show details for one specific process by PID
ps aux | grep name	Filter process list by name
ps aux | wc -l	Count total number of running processes

Understanding ps aux column headers:

Header	Meaning
USER	Which user owns this process
PID	Process ID — unique identifier
%CPU	Percentage of CPU currently being used
%MEM	Percentage of RAM currently being used
VSZ	Virtual memory size in KB — total memory reserved (including unused)
RSS	Resident Set Size — actual physical RAM in use right now (more accurate than VSZ)
TTY	Terminal — ? means background service with no terminal
STAT	Process state — S=sleeping R=running Z=zombie T=stopped s=session leader
START	When the process started
TIME	Total CPU time consumed since start
COMMAND	The command that launched this process

Examples on nezuko:

```
ps # Show only your current terminal's processes
ps aux # Show ALL processes on nezuko
ps aux | grep jellyfin # Find the Jellyfin process
ps aux | grep scottp # Show all processes owned by scottp
ps aux --sort=-%mem | head -5 # Top 5 memory-hungry processes
ps aux --sort=-%cpu | head -5 # Top 5 CPU-hungry processes
ps aux | wc -l # Count total running processes on nezuko
ps -p 2091 # Show details for process ID 2091 (sshd on nezuko)
```

Every flag explained:

Flag	Full explanation
a	Show processes from ALL users, not just your own
u	Use user-oriented format — shows USER, %CPU, %MEM columns
x	Include processes NOT attached to a terminal — catches all daemons and background services
--sort=-%mem	Sort by memory usage descending (- means descending, %mem is the column)
--sort=-%cpu	Sort by CPU usage descending
-p PID	Show only the process with this specific PID

Command: pgrep

`pgrep`

Search for processes by name and return their PIDs

What it does:
pgrep is a cleaner alternative to 'ps aux | grep name' when you just need the PID. It searches running processes by name and returns matching PIDs — useful in scripts where you need to act on a process ID.

Common options:

Option	What it does
pgrep name	Return PIDs of processes matching name
pgrep -l name	Return PIDs AND process names
pgrep -u scottp	Return PIDs of all processes owned by scottp
pgrep -a name	Return PID and full command line

```
pgrep sshd # Get PID of the SSH daemon on nezuko
pgrep -l docker # Get PID and name of docker processes
pgrep -u scottp # All PIDs owned by scottp
pgrep jellyfin # Find Jellyfin's PID
```

Command: top

`top`

Real-time interactive process viewer

What it does:
top shows a live, continuously updating view of all running processes, sorted by CPU usage by default. It also shows system-wide stats at the top — uptime, load average, total tasks, CPU usage, and memory usage. Unlike ps, top refreshes every few seconds so you can watch things change.

Interactive keys while top is running:

Key	What it does
q	Quit top and return to terminal
k	Kill a process — top will ask for the PID then the signal
M	Sort by memory usage (most memory hungry at top)
P	Sort by CPU usage (default)
u	Filter by user — shows only processes owned by a specific user
h	Show help screen with all key bindings
1	Toggle showing individual CPU cores

```
top # Launch top on nezuko
top -u scottp # Launch top showing only scottp's processes
top -p 2091 # Watch a specific process by PID
```

Command: htop

`htop`

Enhanced interactive process viewer — more visual than top

What it does:
htop is an improved version of top with a colour-coded visual interface, mouse support, and easier navigation. It shows CPU and memory as visual bars at the top rather than just numbers. It is not installed by default on all systems but is available on nezuko.

Interactive keys in htop:

Key	What it does
F9	Kill the selected process — shows signal menu
F6	Sort by a chosen column
F5	Toggle tree view — shows parent/child process relationships
F4	Filter — search for processes by name
F10 or q	Quit htop
Arrow keys	Navigate the process list
Space	Tag a process (for bulk operations)

```
htop # Launch htop on nezuko
sudo apt install htop # Install if not present
```

■ htop may not be available in minimal Docker containers or embedded systems. Always know how to use ps and top as fallbacks — those are available everywhere.

Commands: kill / killall / pkill

`kill killall pkill`

Send signals to processes to stop or control them

What they do:
These three commands all stop processes but work differently. kill targets a specific PID. killall targets all processes with a given name. pkill targets processes by name pattern and is more flexible.

All options:

Command	What it does
kill PID	Send SIGTERM (signal 15) to PID — polite request to stop
kill -9 PID	Send SIGKILL (signal 9) to PID — force kill, cannot be ignored
kill -15 PID	Same as kill PID — explicit SIGTERM
kill -1 PID	Send SIGHUP — reload config without killing (useful for services)
kill -l	List all available signal names and numbers
killall name	Kill all processes named exactly 'name'
killall -9 name	Force kill all processes named 'name'
pkill name	Kill processes whose name matches the pattern
pkill -f pattern	Kill processes whose full command line matches pattern
pkill -u scottp	Kill all processes owned by user scottp

Examples on nezuko:

```
kill 3421 # Politely ask process 3421 to stop
kill -9 3421 # Force kill process 3421 — no mercy
killall sleep # Kill all sleep processes (useful after testing &)
pkill -f 'python3 trading.py' # Kill a specific Python script by name
kill -1 2091 # Tell sshd to reload its config (PID 2091 on nezuko)

# Find PID first then kill:
pgrep jellyfin # Find Jellyfin's PID
kill $(pgrep jellyfin) # Kill it in one line using command substitution
```

■ Always try kill (SIGTERM) first and give the process a few seconds. Only use kill -9 if SIGTERM is ignored — force kill skips cleanup and can corrupt data.

Background Process Control: & jobs fg bg Ctrl+Z

`& jobs fg bg`

Run and manage background processes in your terminal session

What they do:
By default a command takes over your terminal until it finishes. The & operator sends it to the background so you keep your prompt. jobs lists background tasks, fg brings one back, bg resumes a suspended one.

Command	What it does
command &	Run command in background — returns prompt immediately, shows [job] PID
jobs	List all background jobs in current terminal session
fg	Bring the most recent background job to the foreground
fg %1	Bring specific job number 1 to the foreground
bg	Resume a suspended process in the background
bg %1	Resume specific job number 1 in the background
Ctrl+Z	Suspend (pause) the currently running foreground process
Ctrl+C	Send SIGINT to foreground process — interrupt/stop it

Examples on nezuko:

```
sleep 60 & # Run sleep in background – safe demo
jobs # See [1]+ Running sleep 60 &
fg %1 # Bring sleep back to foreground
# Now press Ctrl+Z # Suspend it
bg %1 # Resume it in the background
kill %1 # Kill background job 1 by job number

# Real world use on nezuko:
sudo apt upgrade & # Run upgrade in background, keep working
python3 collector.py & # Run trading collector in background
jobs # Check all background jobs
```

■ ADDITIONAL: nohup — Why & alone is not enough for long running tasks Added because: background jobs started with & are tied to your terminal session. When you close SSH, all background jobs die. For anything you want to survive after logout use nohup or tmux instead. nohup command & — runs command immune to hangup signal, output goes to nohup.out nohup python3 collector.py & # Survives SSH disconnect on nezuko tmux is the better solution for interactive sessions — see tmux notes.

9.2 System Information Commands

These commands give you a picture of your hardware and system state — essential for troubleshooting and understanding what nezuko is doing.

`uname`

Print system/kernel information

What it does:
uname prints information about the running kernel and system architecture. Useful for confirming what OS version and kernel you are running.

Option	What it shows
uname	Just the kernel name (Linux)
uname -a	All information — kernel name, hostname, version, architecture
uname -r	Kernel release version only
uname -m	Machine architecture — x86_64, aarch64 etc
uname -n	Hostname only

```
uname -a # Full system info on nezuko
# Example output:
# Linux Nezuko 6.x.x-generic #xx-Ubuntu SMP x86_64 GNU/Linux
uname -r # Just the kernel version
uname -m # Check if x86_64 (nezuko laptop) or aarch64 (Pi)
```

`uptime`

Show how long the system has been running

What it does:
uptime shows the current time, how long the system has been up, how many users are logged in, and the load averages for the last 1, 5, and 15 minutes.

Understanding load averages:
Load average represents the average number of processes waiting to run. On a single core system, 1.0 means fully loaded. On a 4-core system, 4.0 means fully loaded. Below the core count is healthy — above it means the system is struggling.

```
uptime
# Example output on nezuko:
# 14:30:22 up 4 days, 3:42, 2 users, load average: 0.00, 0.01, 0.05
# [redacted]
# how long running users 1min 5min 15min averages

who -b # Show exact date and time of last boot
```

`free`

Display memory usage

What it does:
free shows total, used, free, and available RAM plus swap space. The most important column is 'available' — not 'free'. Available includes memory currently used for cache that can be freed instantly if needed.

Option	What it does
free	Show memory in bytes (not very readable)
free -h	Human readable — shows MB/GB automatically — always use this
free -m	Show in megabytes
free -g	Show in gigabytes
free -s 5	Update every 5 seconds continuously

Understanding the columns:

Column	What it means
total	Total physical RAM installed
used	RAM currently in use by processes
free	RAM not used at all — usually low and misleading
shared	RAM used by tmpfs (shared memory)
buff/cache	RAM used for disk caching — Linux uses spare RAM for this automatically
available	RAM available for new processes — this is the real useful number

```
free -h # Check nezuko's memory in human readable format
free -h -s 5 # Watch memory update every 5 seconds
```

■ Do not panic if 'free' shows very little memory. Linux intentionally uses spare RAM for disk caching. The 'available' column is the one that matters — if available is healthy, your system is fine.

`lscpu`

Display CPU architecture information

What it does:
lscpu reads CPU information from /proc/cpuinfo and presents it in a clean readable format. Shows architecture, number of cores, threads, clock speed, cache sizes, and virtualisation support.

Option	What it does
lscpu	Show all CPU information
lscpu | grep -E '^(CPU\(s\)|Model name)'	Show just core count and model name
lscpu | grep MHz	Show CPU clock speed
lscpu | grep Virtualization	Check if CPU supports virtualisation (needed for Proxmox/VMs)

```
lscpu # Full CPU info on nezuko
lscpu | grep -E '^(CPU\(s\)|Model name)' # Just the essentials
lscpu | grep Virtualization # Check VM support for Beelink Proxmox setup
```

`lsblk`

List block devices (disks and partitions)

What it does:
lsblk shows all storage devices attached to the system — hard drives, SSDs, USB drives, NVMe drives — and their partitions. It shows the device name, size, type, and where it is mounted.

Option	What it does
lsblk	Show all block devices in tree format
lsblk -f	Show filesystem type and UUID for each partition
lsblk -o NAME,SIZE,TYPE,MOUNTPOINT	Show specific columns only
lsblk -d	Show disks only, no partitions

```
lsblk # Show all disks and partitions on nezuko
lsblk -f # Show filesystem types (ext4, vfat etc)

# When Pi NVMe is set up — use lsblk to confirm it is recognised:
lsblk # Should show nvme0n1 device
```

`df`

Report disk space usage of mounted filesystems

What it does:
df shows how much space is used and available on each mounted filesystem. Unlike lsblk which shows physical devices, df shows logical usage per mount point.

Option	What it does
df -h	Human readable sizes — always use this
df -h /	Show space on root filesystem only
df -T	Include filesystem type column
df -i	Show inode usage instead of disk space

```
df -h # Check all disk space on nezuko
df -h / # Check root partition specifically
df -h /media # Check media folder space (Jellyfin content)
```

9.3 Service Management with systemd

What is systemd?

systemd is the **init system** on modern Linux — it is PID 1, the first process that starts when the kernel boots, and the parent of everything else. Think of it as the manager of all services on your system. It starts services at boot, manages dependencies between them, restarts them if they crash, and collects their logs.

Key systemd concepts:

Term	Meaning
Unit	The basic object systemd manages — a service, socket, timer, mount point etc
Service unit	A background process managed by systemd (ssh.service, cron.service etc)
enabled	Service will start automatically at boot
disabled	Service will NOT start at boot
active (running)	Service is currently running
inactive (dead)	Service is not currently running
failed	Service tried to start but crashed

`systemctl`

Control and inspect systemd services

What it does:
systemctl is the primary tool for interacting with systemd. It lets you start, stop, restart, enable, disable, and inspect services. Most service commands require sudo.

All key options:

Command	What it does
systemctl status name	Show current status, recent logs, PID, memory for a service
sudo systemctl start name	Start a service right now (does not affect boot behaviour)
sudo systemctl stop name	Stop a running service right now
sudo systemctl restart name	Stop then start a service — applies config changes
sudo systemctl reload name	Reload config without full restart (if service supports it)
sudo systemctl enable name	Make service start automatically at boot
sudo systemctl disable name	Stop service starting at boot
sudo systemctl enable --now name	Enable AND start immediately in one command
systemctl is-enabled name	Check if service is set to start at boot
systemctl is-active name	Check if service is currently running
systemctl list-units --type=service	List all service units
systemctl list-units --type=service --state=running	List only currently running services

Examples on nezuko:

```
systemctl status ssh # Check SSH service — see PID, uptime, recent logs
systemctl status cron # Check cron scheduler
systemctl status docker # Check Docker

sudo systemctl stop cron # Stop cron
systemctl status cron # Confirm stopped (inactive)
sudo systemctl start cron # Start it again
sudo systemctl restart ssh # Restart SSH after config change

sudo systemctl enable cron # Make cron start at boot
sudo systemctl disable cron # Stop cron starting at boot
systemctl is-enabled ssh # Returns 'enabled' or 'disabled'

systemctl list-units --type=service --state=running # All running services
```

■ Restarting SSH while connected via SSH is risky — your session may drop. Use 'sudo systemctl reload ssh' where possible, or make sure Tailscale is running so you can reconnect via 100.76.26.45 if needed.

`journalctl`

View and search systemd logs

What it does:
journalctl reads the systemd journal — the central log store for all services managed by systemd on modern Ubuntu. This replaces /var/log/auth.log and similar files on Ubuntu 26.04. All service output, errors, and events are stored here in binary format and queried via journalctl.

All key options:

Command	What it does
journalctl	Show all logs from newest boot — press q to quit
journalctl -u name	Show logs for a specific service only
journalctl -f	Follow logs in real time — like tail -f for system logs
journalctl -b	Show logs since last boot only
journalctl -b -1	Show logs from the previous boot
journalctl --since today	Show logs from today only
journalctl --since '1 hour ago'	Show logs from last hour
journalctl --since '2026-06-08 10:00'	Show logs since specific date/time
journalctl -u name --since today	Combine service filter with time filter
journalctl -n 50	Show last 50 log lines
journalctl -p err	Show only error level messages and above
journalctl | grep pattern	Search logs for a specific pattern

Examples on nezuko:

```
journalctl -u ssh --since today # SSH logins today
journalctl -u ssh | grep Accepted # All successful SSH logins ever
journalctl -u ssh | grep 192.168.1.188 # Logins from your Windows PC
journalctl -u cron --since today # What cron ran today
journalctl -u docker -f # Watch Docker logs live
journalctl -b # Everything since nezuko last booted
journalctl -p err -b # Errors since last boot
journalctl --since '1 hour ago' -f # Recent logs, follow in real time
```

Challenge — Explore Processes and Services on nezuko

All commands needed to complete the challenge:

```
# Count all running processes:
ps aux | wc -l

# Find process using most memory:
ps aux --sort=-%mem | head -5
# Or launch htop, press F6, choose MEM%

# Check SSH service status:
systemctl status ssh

# Check when nezuko last booted:
uptime
who -b

# Check available memory:
free -h
# Look at the 'available' column — not 'free'
```

Quick Reference — All Commands

Command	What it does
ps	Snapshot of your terminal's processes
ps aux	Snapshot of ALL processes on the system
ps aux --sort=-%mem | head -5	Top 5 memory users
pgrep name	Get PID of process by name
top	Real-time process viewer (built in everywhere)
htop	Enhanced real-time viewer (needs installing)
kill PID	Send SIGTERM to process — polite stop
kill -9 PID	Send SIGKILL — force stop, cannot be ignored
killall name	Kill all processes with this name
pkill -f pattern	Kill processes matching command pattern
command &	Run command in background
jobs	List background jobs
fg %1	Bring job 1 to foreground
bg %1	Resume job 1 in background
Ctrl+Z	Suspend foreground process
Ctrl+C	Interrupt/stop foreground process
uname -a	Full kernel and system info
uptime	System uptime and load averages
free -h	Memory usage in human readable format
lscpu	CPU architecture and core info
lsblk	Disks and partitions
df -h	Disk space usage
systemctl status name	Service status and recent logs
sudo systemctl start/stop/restart name	Control a service
sudo systemctl enable/disable name	Control boot behaviour
systemctl list-units --type=service	List all services
journalctl -u name	Logs for a specific service
journalctl -f	Follow all logs in real time
journalctl -b	Logs since last boot
journalctl --since today	Today's logs

■ Linux+ Exam Tips: Know ps aux columns especially STAT codes (S R Z T). Know SIGTERM (15) vs SIGKILL (9) — SIGKILL cannot be ignored. Know that systemctl enable affects boot behaviour, start/stop affects right now. Know journalctl replaces /var/log on modern Ubuntu. Know PID 1 is always systemd on modern Linux. Know that & runs in background but dies on SSH disconnect — use nohup for persistence.